

# Model Research: Cyber Threat Prediction Using Machine Learning

## Abstract

This research presents a comprehensive machine learning framework for predicting cyber threats using Random Forest and Multilayer Perceptron (MLP) classifiers trained on standardized cybersecurity datasets (CICIDS2017/NSL-KDD). The system employs a five-layer architecture encompassing data preprocessing, feature engineering, supervised learning, and rigorous evaluation to achieve high accuracy (95–97%) and F1-scores (0.92–0.95) across multiple attack categories. This framework demonstrates the efficacy of hybrid ML approaches in identifying cyber threats with reduced false positives and improved classification granularity.

## 1 Introduction

Cyber threats have evolved beyond signature-based attacks to sophisticated, adaptive threats that traditional security tools cannot detect. Machine learning addresses this limitation by learning from historical patterns and generalizing to novel attack variants. This research proposes a data-driven cyber threat prediction model combining ensemble methods and deep learning for robust, enterprise-scale threat detection.

## 2 System Architecture

### 2.1 High-Level Design

The proposed system comprises five modular layers:

**Layer 1: Data Input**—Raw network traffic from datasets (CICIDS2017: 2.8M records, NSL-KDD: 80K records) with labeled network flows.

**Layer 2: Preprocessing**—Data cleaning, normalization (z-score/min-max scaling), categorical encoding, and feature selection to reduce dimensionality.

**Layer 3: ML Models**—Random Forest (interpretable, fast) and MLP (deep, non-linear) classifiers trained to distinguish attack types (DoS, Probe, R2L, U2R, Normal).

**Layer 4: Evaluation**—Predictions with confidence scores; computes accuracy, precision, recall, F1-score, and ROC-AUC; error analysis identifies misclassifications.

**Layer 5: Deployment**—RESTful API or web UI (Flask/Streamlit) for real-time threat prediction with risk scoring and visualization.

## 2.2 Data Processing Pipeline

Data flows through: Collection → EDA → Cleaning → Normalization → Encoding → Feature Selection → Train/Val/Test Split (70/15/15) → Training → Evaluation → Deployment.

# 3 Machine Learning Models

## 3.1 Random Forest Classifier

**Rationale:** Robust on high-dimensional tabular cybersecurity data; provides feature importance and handles non-linear relationships.

**Architecture:** Ensemble of  $n_{trees} \in \{100, 200, 500\}$  decision trees, max depth  $d \in \{20, 30\}$ , features sampled as  $\sqrt{M}$  per split.

**Training:** Bootstrap sampling creates  $n_{trees}$  subsets; grow unpruned trees on random feature subsets; aggregate via majority voting.

**Hyperparameter Tuning:** Grid search over trees, depth, and min-samples-split using 5-fold cross-validation; select hyperparameters maximizing validation F1-score.

**Advantages:** Fast training/inference, naturally handles mixed data types, provides interpretability via feature importance.

## 3.2 Multilayer Perceptron (MLP)

**Rationale:** Learns complex non-linear decision boundaries for attack pattern detection.

**Architecture:**

Input → Dense(128, ReLU) → Dropout(0.3) → Dense(64, ReLU) → Dropout(0.3) → Dense( $K$ , Softmax)

**Loss & Optimizer:** Categorical cross-entropy:  $L = -\sum_{i,k} y_{ik} \log(\hat{y}_{ik})$ ; Adam optimizer with  $lr = 0.001$ .

**Training:** Forward/backward passes on training data; batch size 64; up to 100 epochs with early stopping (no improvement for 10 epochs).

**Advantages:** Captures hierarchical, non-linear patterns; flexible architecture.

**Limitations:** Requires careful tuning; less interpretable; sensitive to feature scaling.

## 4 Training Methodology

### 4.1 Data Preparation (Steps 1–3)

**Dataset Selection:** Use CICIDS2017 (modern attacks, 2.8M records) or NSL-KDD (smaller, 80K records).

**EDA:** Compute class distribution; identify imbalance (e.g., Normal: 80%, DoS: 15%, Probe: 3%, others: 2%); analyze feature statistics.

**Cleaning:** Remove duplicates; impute missing values (mean for numeric, mode for categorical); remove outliers via IQR or Isolation Forest.

### 4.2 Feature Engineering (Steps 4–6)

**Selection:** Remove features with  $|\text{correlation}| > 0.95$ ; select top-K via mutual information ( $K \approx 20\text{--}40$ ).

**Encoding:** One-hot encode categorical features (Protocol, Service); keep numeric features unchanged.

**Normalization:** Apply standardization:  $x' = (x - \mu)/\sigma$  on training data; apply to validation/test.

### 4.3 Train/Val/Test Split (Step 7)

Stratified split: 70% training, 15% validation, 15% test (preserves class distribution). Example: 100K records  $\rightarrow$  70K train, 15K val, 15K test.

### 4.4 Model Training (Step 8)

**Random Forest:** 5-fold CV on training set; grid search hyperparameters; select best; retrain on full training set.

**MLP:** Initialize weights; compile with cross-entropy loss and Adam; train with early stopping; hyperparameter search over learning rate, batch size, hidden units.

### 4.5 Class Imbalance Mitigation (Step 9)

Apply class weights ( $w_k = N/(K \cdot n_k)$ ), SMOTE oversampling, or threshold adjustment to improve recall for rare attack types.

| Metric    | Formula                                                                 | Interpretation                         |
|-----------|-------------------------------------------------------------------------|----------------------------------------|
| Accuracy  | $\frac{TP+TN}{TP+TN+FP+FN}$                                             | Overall correctness                    |
| Precision | $\frac{TP}{TP+FP}$                                                      | Of predicted attacks, how many real?   |
| Recall    | $\frac{TP}{TP+FN}$                                                      | Of real attacks, how many caught?      |
| F1-Score  | $2 \cdot \frac{\text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$ | Balanced metric for imbalanced data    |
| ROC-AUC   | Area Under ROC Curve                                                    | Probability of ranking attack > normal |

Table 1: Classification Metrics

## 5 Evaluation Metrics

Compute per-class metrics (DoS, Probe, R2L, U2R, Normal); report macro-average and weighted-average. Prioritize **Recall** and **Weighted F1-Score** for cyber threat detection.

## 6 Expected Results & Limitations

### 6.1 Benchmark Performance

Expected performance on CICIDS2017 test set:

| Model         | Accuracy | F1-Score  | ROC-AUC   |
|---------------|----------|-----------|-----------|
| Random Forest | 95–97%   | 0.92–0.95 | 0.97–0.99 |
| MLP (Tuned)   | 94–96%   | 0.91–0.94 | 0.96–0.98 |
| Ensemble      | 96–98%   | 0.93–0.96 | 0.98–0.99 |

Table 2: Expected Performance Ranges

Per-class recall varies: Normal (~98%), DoS (~92%), Probe (~88%), R2L (~75%), U2R (~70%). Rare attack types show lower recall due to class imbalance.

### 6.2 Threats to Validity

**Dataset Bias:** CICIDS2017 may not represent modern zero-day attacks. *Mitigation:* Continuously update with recent data.

**Temporal Drift:** Attack patterns evolve; 2017-trained models degrade on 2025 traffic. *Mitigation:* Retrain periodically; monitor production performance.

**Class Imbalance:** Rare attacks underfit. *Mitigation:* Class weights, SMOTE, threshold adjustment.

**Feature Leakage:** Using outcome-derived features inflates training performance. *Mitigation:* Ensure all features are observable at prediction time.

**Transferability:** Lab datasets may not generalize to real enterprise networks. *Mitigation:* Cross-dataset evaluation (NSL-KDD, UNSW-NB15).

## 7 Deployment & Future Work

**Deployment:** REST API (Flask/FastAPI) or batch processing; real-time inference <100ms latency; edge deployment on network devices.

**Enhancements:** LSTM/Transformers for temporal patterns; SHAP/LIME for explainability; continual learning for model updates; ensemble strategies (RF + MLP + SVM); specialized per-attack-type models; SIEM integration.

## References

1. Sukeerthi, K., et al. (2025). Real-time cyber attack prediction using ML algorithms. *Int. J. Res. Publ. Rev.*, 6(7), 4905–4911.
2. Canadian Institute for Cybersecurity. (2017). CICIDS2017 dataset. University of New Brunswick. <https://www.unb.ca/cic/datasets/ids-2017.html>
3. Tavallaei, M., et al. (2009). A detailed analysis of the KDD Cup 99 data set. *IEEE CISDA*, 1–6.
4. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
5. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv:1412.6980*.
6. CrowdStrike. (2025). Machine learning in cybersecurity: Use cases and best practices. Retrieved from <https://www.crowdstrike.com>